Module 1: Introduction & Communication Mechanisms

1.1 What Exactly Is a Distributed System?

A **distributed system** is a collection of independent computers, connected by a network, that appears to its users as a single coherent system. The word "independent" matters: each machine has its own memory, its own processor, and can fail on its own without necessarily bringing down the others. The system's job is to hide this messy reality and present one clean, unified experience.

Three properties make distributed systems fundamentally harder to reason about than a single computer:

- **Concurrency:** Many machines are doing things at the same time, so operations can interleave in unpredictable ways.
- **No global clock:** Each machine has its own local clock, and clocks drift, so you cannot simply trust timestamps to order events across machines.
- **Independent failure:** A machine can crash, or a network link can drop, while the rest of the system keeps running — and the other machines may not immediately know about it.

Aspect	Centralized System	Distributed System
Control	Single point of control	Control is spread across nodes
Failure impact	One crash can halt everything	Other nodes can keep working
Clock	One shared system clock	Each node has its own clock
Scaling	Scale up (bigger machine)	Scale out (add more machines)

1.1.1 Why Do We Build Distributed Systems?

- **Resource Sharing:** Facilitates the sharing of hardware peripherals, centralized storage, and data processing power. Example: a university's printers and file servers are shared by thousands of students from one shared infrastructure.
- **Computation Speedup:** Allows for parallel processing where complex tasks are divided across multiple machines. Example: rendering a Pixar movie frame-by-frame across a render farm of hundreds of machines.
- **Reliability:** Eliminates single points of failure. If one node goes down, the system redistributes the workload. Example: Netflix keeps streaming even when an entire data center goes offline.
- **Communication:** Enables robust data exchange across massive geographic distances. Example: a video call between Lagos and London, hopping across dozens of routers in milliseconds.

1.1.2 Transparency: The Illusion of a Single System

For a distributed system to "feel" like one machine, it must hide distribution-related details from the user. Computer scientists break this hiding down into specific types of **transparency**:

Transparency Type	What It Hides From the User
Access	Whether data is local or remote — you use the same commands either way.
Location	Where a resource is physically located.
Migration	That a resource might move to a new location while in use.
Relocation	That a resource may move even while being actively accessed.
Replication	That multiple copies of a resource exist.
Concurrency	That several users may be sharing the same resource simultaneously.
Failure	That a component failed and was recovered or rerouted around.
Persistence	Whether a resource lives in memory or on disk.

■ The Web Architecture Analogy

Imagine a modern web application designed for a university. The frontend (built with React or Next.js) lives on one server, the application logic (Node.js or PHP) is hosted on another, and the database (MySQL) runs on a completely separate, optimized server. To a student logging in to check their grades, it feels like a single, unified website. They are entirely unaware of the complex, distributed architecture working together behind the scenes — that is *access* and *location* transparency in action.

1.2 Communication Mechanisms & Protocols

Underlying all distributed architectures are robust networking protocols, generally following layered models like OSI or TCP/IP. In practice, almost everything you need to know for this course comes down to a choice between two transport protocols.

Feature	TCP/IP	UDP
Connection	Connection-oriented (handshake first)	Connectionless (just send)
Delivery guarantee	Guaranteed — retransmits lost packets	Not guaranteed — packets may be lost
Ordering	Packets arrive in order	No ordering guarantee
Speed	Slower, due to overhead	Faster, minimal overhead
Typical use	Web browsing, file transfer, email	Video calls, live streaming, online gaming

■ Choosing the Right Protocol

Downloading a PDF must use TCP: if even one packet goes missing, the file becomes corrupted, so guaranteed delivery matters more than speed. A live video call typically uses UDP: if one video frame is lost, it is far better to skip it and keep the call moving in real time than to pause and wait for a retransmission.

1.3 Remote Procedure Call (RPC)

RPC is a powerful abstraction that allows a program to execute a subroutine on another computer without having to explicitly code the network interaction. The programmer simply calls a function, and RPC machinery makes a network call look exactly like a local one.

Marshalling is the process of packaging function parameters (e.g., numbers, strings, objects) into a format that can be sent over a network — essentially converting them into a byte stream. **Unmarshalling** is the reverse: turning that byte stream back into usable parameters on the receiving end. This packaging/unpackaging work is handled automatically by generated **stub functions**, so the programmer never has to write networking code by hand.

RPC CALL & RETURN FLOW ARCHITECTURE

```
CLIENT MACHINE                                  SERVER MACHINE
------------------------------                 ------------------------------
1. User calls add(5, 7)
2. Client Stub marshals data
3. OS sends network packet     --- Request --->   4. OS receives packet
                                                    5. Server Stub unmarshals
                                                    6. Server executes add()
                                                    7. Returns result 12
                                                    8. Stub marshals result
10. OS receives result         <--- Reply ------   9. OS sends reply packet
11. Client Stub unmarshals
12. Returns 12 to User
```

Notice that from the user's point of view, steps 2 through 11 are completely invisible. They called `add(5, 7)` and got back `12`, exactly as if the function lived on their own machine.

Call Semantics: What Happens If a Message Is Lost?

Because networks are unreliable, RPC systems must decide what to do if a request or reply goes missing. This is described using **call semantics**:

Semantics	Behavior	Risk
At-least-once	Keeps retrying until it gets a reply	The call might run more than once
At-most-once	Never retries automatically	The call might not run at all
Exactly-once	Ideal — runs exactly one time	Hard to guarantee in practice; needs careful design

1.4 Remote Method Invocation (RMI) & Stream-Oriented Communication

While RPC is procedural (it calls plain functions), RMI brings the same remote-execution concept to object-oriented environments (like Java). An object living in one JVM (Java Virtual Machine) can seamlessly invoke methods on an object living in a completely different JVM, on a different machine.

RMI relies on a **stub** on the client side (representing the remote object locally) and a **skeleton** on the server side (which receives the call and invokes the real object). A naming/registry service lets clients look up remote objects by name, similar to how DNS lets you look up servers by domain name (see Module 3).

Stream-Oriented Communication handles continuous media flow, such as audio and video. Unlike the discrete, one-shot request-reply nature of RPC, streams require strict timing guarantees, known as **Quality of Service (QoS)**, to prevent jitter and latency.

QoS Parameter	What It Measures
Bandwidth	How much data can be transferred per second.
Delay (latency)	How long a packet takes to travel from sender to receiver.
Jitter	The variation in delay between packets — the enemy of smooth video/audio.
Reliability	The probability that data arrives without loss or corruption.

> ■ **Key Takeaway**
>
> RPC and RMI both try to make remote operations feel local, but streaming media is different — it cares less about perfect reliability and more about steady, predictable timing.

Glossary of Key Terms

Term	Meaning
Marshalling	Packaging parameters into a transmittable byte stream.
Stub	A local proxy that forwards a call to a remote machine.
Skeleton	The server-side counterpart of a stub in RMI; receives and dispatches calls.
QoS	Quality of Service — guarantees about bandwidth, delay, jitter and reliability.
Transparency	Hiding some aspect of distribution from the end user.

What You Should Know

- The core definition of a distributed system and the three properties that make it hard (concurrency, no global clock, independent failure).
- The eight types of transparency and what each one hides from the user.
- The distinction between connection-oriented (TCP) and connectionless (UDP) protocols, and when to use each.
- The step-by-step lifecycle of an RPC call, particularly the roles of the client/server stubs in marshalling/unmarshalling data.
- The difference between at-least-once, at-most-once, and exactly-once call semantics.
- How RMI translates RPC concepts into object-oriented programming, and why streaming needs QoS instead of strict reliability.